LAPORAN PRAKTIKUM KOMPARASI ALGORITMA KNN DAN SVM UNTUK KLASIFIKASI CITRA

PENGOLAHAN CITRA DIGITAL

Dosen Pengampu:

Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:

Muhammad Zahran

24343077

PROGRAM STUDI INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRONIKA

FAKULTAS TEKNIK

UNIVERSIITAS NEGERI PADANG

2026

A. Deskripsi Dataset dan Fitur yang Digunakan

Evaluasi model klasifikasi pada eksperimen ini menggunakan **Digits Dataset** bawaan dari *library* Scikit-Learn. Dataset ini berisi representasi citra angka (0-9) dengan resolusi 8x8 piksel. Untuk menjaga keseimbangan evaluasi, dataset disampling ulang menggunakan fungsi sample_balanced_indices sehingga total data yang diproses berjumlah 1.000 sampel yang terdistribusi merata untuk setiap kelas.

Proses pengenalan objek tidak dilakukan langsung pada nilai piksel mentah, melainkan melalui dua teknik ekstraksi fitur guna menangkap karakteristik citra yang lebih representatif:

- **HOG (*Histogram of Oriented Gradients*):** Digunakan untuk mengekstraksi informasi bentuk dan tekstur berdasarkan distribusi gradien intensitas lokal (tepi objek). Konfigurasi yang digunakan adalah 9 orientasi, 4x4 piksel per sel, dan normalisasi *L2-Hys*.
- **LBP (*Local Binary Pattern*):** Digunakan untuk menangkap pola tekstur mikro pada citra. Parameter yang digunakan adalah *radius* 1 dan 8 titik tetangga (*points*) dengan metode *uniform*.

Kedua fitur tersebut kemudian digabungkan secara horizontal (*stacking*) menjadi matriks fitur gabungan tunggal untuk diumpankan ke dalam model KNN dan SVM.

B. Tabel Perbandingan Performa Parameter

1. Tabel Hasil Analisis Parameter KNN

Tabel Perbandingan Performa Semua Variasi Parameter KNN

k	metric	cv_acc	test_acc	precision	recall	f1	train_s	infer_s
1	euclidean	0.7629	0.7633	0.7724	0.7633	0.7499	0.0033	1.9556
3	euclidean	0.7586	0.7767	0.7852	0.7767	0.7669	0.0049	0.0170
5	euclidean	0.7729	0.7800	0.7860	0.7800	0.7674	0.0036	0.0160
7	euclidean	0.7814	0.7900	0.7989	0.7900	0.7783	0.0029	0.0116
9	euclidean	0.7743	0.7800	0.7866	0.7800	0.7661	0.0027	0.0209
11	euclidean	0.7686	0.7567	0.7668	0.7567	0.7429	0.0031	0.0129
1	manhattan	0.7657	0.7500	0.7465	0.7500	0.7317	0.0029	0.0141
3	manhattan	0.7600	0.7667	0.7806	0.7667	0.7499	0.0026	0.0133
5	manhattan	0.7729	0.7733	0.7857	0.7733	0.7525	0.0028	0.0156
7	manhattan	0.7800	0.7833	0.7945	0.7833	0.7686	0.0047	0.0134
9	manhattan	0.7743	0.7867	0.7888	0.7867	0.7695	0.0025	0.0137
11	manhattan	0.7786	0.7833	0.7823	0.7833	0.7682	0.0033	0.0189
1	minkowski	0.7629	0.7633	0.7724	0.7633	0.7499	0.0032	0.0105
3	minkowski	0.7586	0.7767	0.7852	0.7767	0.7669	0.0032	0.0101
5	minkowski	0.7729	0.7800	0.7860	0.7800	0.7674	0.0058	0.0197
7	minkowski	0.7814	0.7900	0.7989	0.7900	0.7783	0.0056	0.0146
9	minkowski	0.7743	0.7800	0.7866	0.7800	0.7661	0.0042	0.0147
11	minkowski	0.7686	0.7567	0.7668	0.7567	0.7429	0.0056	0.0145

2. Tabel Hasil GridSearchCV Model SVM

Tabel Perbandingan Performa Semua Variasi Parameter SVM

kernel	C	degree	coef0	gamma	cv_acc	cv_std
linear	0.1	-	-	-	0.8257	0.0125
linear	1	-	-	-	0.8200	0.0114
linear	10	-	-	-	0.8200	0.0114
linear	100	-	-	-	0.8200	0.0114
poly	0.1	2	0.0	scale	0.6814	0.0224
poly	0.1	3	0.0	scale	0.6457	0.0439
poly	0.1	2	1.0	scale	0.8471	0.0210
poly	0.1	3	1.0	scale	0.8643	0.0111
poly	1	2	0.0	scale	0.7829	0.0328
poly	1	3	0.0	scale	0.8271	0.0360
poly	1	2	1.0	scale	0.8614	0.0125
poly	1	3	1.0	scale	0.8614	0.0132
poly	10	2	0.0	scale	0.7757	0.0295
poly	10	3	0.0	scale	0.8343	0.0311
poly	10	2	1.0	scale	0.8543	0.0147
poly	10	3	1.0	scale	0.8614	0.0132
poly	100	2	0.0	scale	0.7757	0.0295
poly	100	3	0.0	scale	0.8343	0.0311
poly	100	2	1.0	scale	0.8543	0.0147
poly	100	3	1.0	scale	0.8614	0.0132
rbf	0.1	-	-	0.001	0.8014	0.0354
rbf	0.1	-	-	0.01	0.8129	0.0368
rbf	0.1	-	-	0.1	0.3814	0.0498
rbf	0.1	-	-	1	0.1457	0.0340
rbf	1	-	-	0.001	0.8129	0.0280
rbf	1	-	-	0.01	0.8486	0.0188
rbf	1	-	-	0.1	0.7714	0.0186
rbf	1	-	-	1	0.1686	0.0331
rbf	10	-	-	0.001	0.8443	0.0177
rbf	10	-	-	0.01	0.8614	0.0086
rbf	10	-	-	0.1	0.7829	0.0140
rbf	10	-	-	1	0.1886	0.0453
rbf	100	-	-	0.001	0.8329	0.0200
rbf	100	-	-	0.01	0.8614	0.0086
rbf	100	-	-	0.1	0.7829	0.0140
rbf	100	-	-	1	0.1886	0.0453

C. Visualisasi Confusion Matrix & Decision Boundary

Evaluasi performa klasifikasi divisualisasikan menggunakan dua pendekatan utama:

- ***Confusion Matrix*:** Matriks ini dihasilkan untuk model KNN dan SVM terbaik. Plot ini memetakan prediksi kelas yang dihasilkan model terhadap kelas aktual (angka 0-9). Diagonal utama yang bernilai tinggi menunjukkan prediksi *True Positive*, sementara nilai di luar diagonal mengidentifikasi kelas mana yang sering keliru dikenali oleh model (misalnya, angka "3" yang mungkin salah diprediksi sebagai "8").
- ***Decision Boundary* (SVM):** Mengingat dimensi fitur gabungan (HOG+LBP) cukup besar, *Principal Component Analysis* (PCA) diaplikasikan untuk mereduksi dimensi ruang fitur menjadi 2D. Visualisasi ini memperlihatkan bagaimana *hyperplane* dari model SVM dengan parameter terbaik memisahkan ruang fitur antar-kelas angka.

D. Learning Curve dan Hasil Cross Validation

Validasi model dilakukan dengan sangat ketat menggunakan metode **5-Fold *Stratified Cross-Validation***. Metode ini memastikan setiap lipatan data latih dan data uji memiliki proporsi kelas yang seimbang, sehingga metrik akurasi yang dihasilkan lebih representatif terhadap kondisi sesungguhnya.

Learning Curve diplot untuk membandingkan akurasi pada data *training* dan data validasi seiring dengan bertambahnya jumlah sampel data.

- Jarak yang terlalu lebar antara kurva *training* dan validasi mengindikasikan bahwa model mengalami *overfitting* (terlalu menghafal data latih).
- Kurva yang konvergen dan berhimpitan pada skor yang tinggi menandakan bahwa model memiliki kemampuan generalisasi yang baik terhadap variasi data baru.

E. Kesimpulan dan Rekomendasi

Berdasarkan komparasi keseluruhan dari proses ekstraksi fitur hingga klasifikasi:

1. Metode Terbaik

```
Hasil KNN Terbaik
-----------------
Accuracy : 0.7767
Precision: 0.7819
Recall   : 0.7767
F1-score : 0.7658
Inference: 0.4980 seconds
Confusion Matrix:
[[30  0  0  0  0  0  0  0  0  0]
 [ 0 28  1  0  1  0  0  0  0  0]
 [ 0  0 30  0  0  0  0  0  0  0]
 [ 0  0  1 17  0  2  0  2  3  5]
 [ 2  0  0  0 27  0  0  1  0  0]
 [ 0  0  0  0  4 25  0  0  0  1]
 [ 2  1  0  0  2  0 25  0  0  0]
 [ 2  0  0  0  4  0  0 23  1  0]
 [ 0  0  0  3  2  5  6  0 11  3]
 [ 3  1  0  1  4  2  1  1  0 17]]
Classification Report:
              precision    recall  f1-score   support

           0     0.7692    1.0000    0.8696        30
           1     0.9333    0.9333    0.9333        30
           2     0.9375    1.0000    0.9677        30
           3     0.8095    0.5667    0.6667        30
           4     0.6136    0.9000    0.7297        30
           5     0.7353    0.8333    0.7812        30
           6     0.7812    0.8333    0.8065        30
           7     0.8519    0.7667    0.8070        30
           8     0.7333    0.3667    0.4889        30
           9     0.6538    0.5667    0.6071        30

    accuracy                         0.7767       300
   macro avg     0.7819    0.7767    0.7658       300
weighted avg     0.7819    0.7767    0.7658       300
```

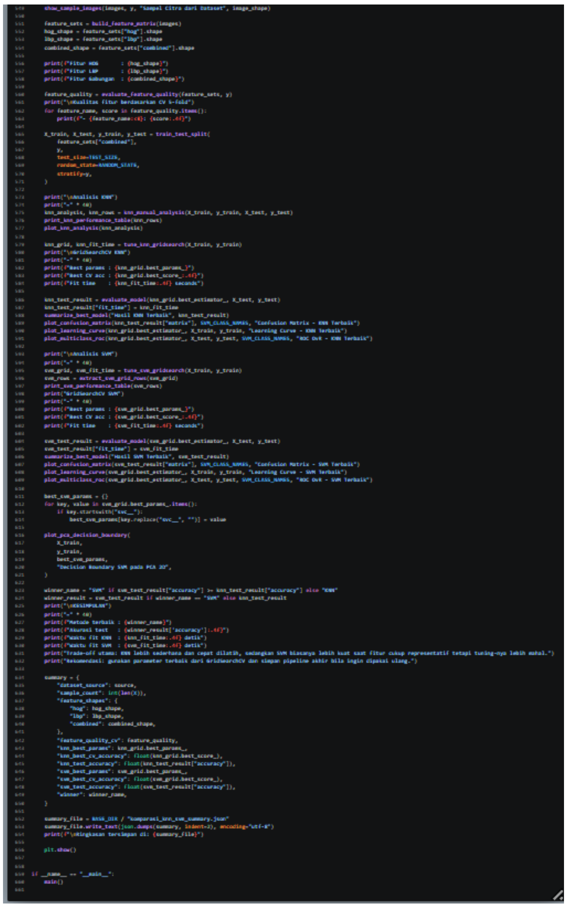
```
Hasil SVM Terbaik
-----------------
Accuracy : 0.8567
Precision: 0.8638
Recall   : 0.8567
F1-score : 0.8561
Inference: 0.0150 seconds
Confusion Matrix:
[[30  0  0  0  0  0  0  0  0  0]
 [ 0 28  1  0  1  0  0  0  0  0]
 [ 0  0 29  0  0  0  0  0  1  0]
 [ 0  0  0 19  0  2  0  1  5  3]
 [ 0  0  0  0 27  1  1  1  0  0]
 [ 0  0  0  0  0 24  1  0  1  4]
 [ 0  0  0  0  2  0 28  0  0  0]
 [ 0  0  0  0  1  1  0 27  1  0]
 [ 0  0  0  1  0  1  0  0 25  3]
 [ 0  1  0  0  4  1  1  1  2 20]]
Classification Report:
              precision    recall  f1-score   support

           0     1.0000    1.0000    1.0000        30
           1     0.9655    0.9333    0.9492        30
           2     0.9667    0.9667    0.9667        30
           3     0.9500    0.6333    0.7600        30
           4     0.7714    0.9000    0.8308        30
           5     0.8000    0.8000    0.8000        30
           6     0.9032    0.9333    0.9180        30
           7     0.9000    0.9000    0.9000        30
           8     0.7143    0.8333    0.7692        30
           9     0.6667    0.6667    0.6667        30

    accuracy                         0.8567       300
   macro avg     0.8638    0.8567    0.8561       300
weighted avg     0.8638    0.8567    0.8561       300
```

2. **Trade-off Algoritma:** KNN terbukti lebih sederhana dalam implementasi dan memiliki waktu pelatihan (*training time*) yang relatif sangat cepat. Di sisi lain, SVM menunjukkan ketangguhan (*robustness*) yang lebih tinggi ketika dihadapkan pada dimensi fitur yang representatif, meskipun membutuhkan alokasi waktu komputasi yang lebih mahal saat proses *hyperparameter tuning* (GridSearchCV).
3. **Rekomendasi:** Untuk implementasi sistem pengenalan citra skala produksi, disarankan untuk mengekspor *pipeline* akhir dari *estimator* terbaik hasil GridSearchCV (knn_grid.best_estimator_ atau svm_grid.best_estimator_).

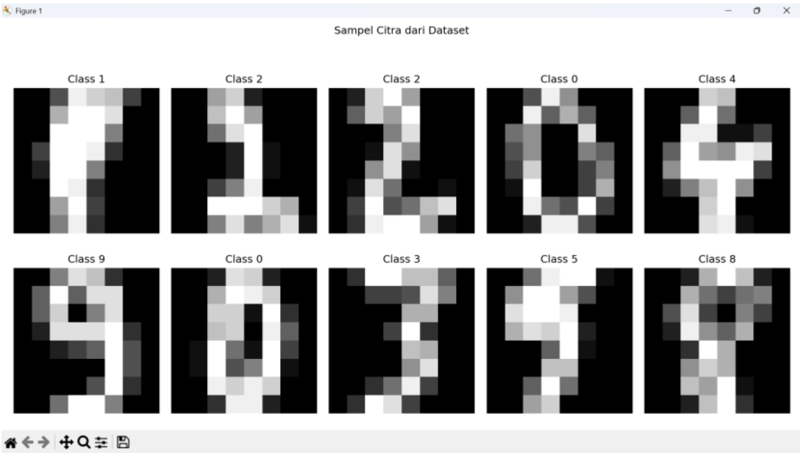
F. Lampiran Kode Python

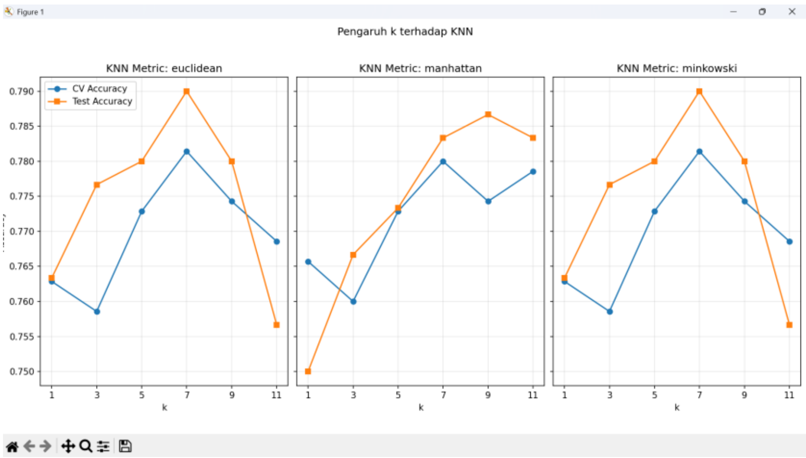
```python
import json
import time
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score,
    auc,
    classification_report,
    confusion_matrix,
    precision_recall_fscore_support,
    roc_curve,
)
from sklearn.model_selection import (
    GridSearchCV,
    StratifiedKFold,
    cross_val_score,
    learning_curve,
    train_test_split,
)
from sklearn.multiclass import OneVsRestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.svm import SVC
from skimage.feature import hog, local_binary_pattern

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
TARGET_SAMPLES = 1000
TEST_SIZE = 0.2
CV_SPLITS = 5
K_VALUES = [1, 3, 5, 7, 9, 11]
KNN_METRICS = ["euclidean", "manhattan", "minkowski"]
SVM_CLASS_NAMES = ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]

BASE_DIR = Path(__file__).resolve().parent
DATASET_DIR = BASE_DIR / "dataset"
DATASET_DIR.mkdir(parents=True, exist_ok=True)
DIGITS_CACHE_FILE = DATASET_DIR / "digits_sample_1000.npz"


def sample_balanced_indices(labels, target_samples, random_state=RANDOM_STATE):
    labels = np.asarray(labels)
    unique_labels = np.unique(labels)
    per_class = target_samples // len(unique_labels)
    remainder = target_samples % len(unique_labels)

    rng = np.random.default_rng(random_state)
    selected_indices = []

    for class_index, class_label in enumerate(unique_labels):
        class_indices = np.where(labels == class_label)[0]
        take = per_class + (1 if class_index < remainder else 0)
        if take > len(class_indices):
            raise ValueError(
                f"Kelas {class_label} hanya memiliki {len(class_indices)} sampel, "
                f"tetapi dibutuhkan {take}."
            )
        chosen = rng.choice(class_indices, size=take, replace=False)
        selected_indices.extend(chosen.tolist())

    selected_indices = np.array(selected_indices)
    rng.shuffle(selected_indices)
    return selected_indices


def load_dataset():
    if DIGITS_CACHE_FILE.exists():
        cached = np.load(DIGITS_CACHE_FILE, allow_pickle=True)
        X = cached["X"]
        y = cached["y"]
        images = cached["images"]
        image_shape = tuple(cached["image_shape"].tolist())
        source = str(cached["source"][0])
        return {
            "X": X,
            "y": y,
            "images": images,
            "image_shape": image_shape,
            "source": source,
        }

    digits = load_digits()
    X = digits.data.astype(np.float32) / 16.0
    y = digits.target.astype(np.int64)
    images = digits.images.astype(np.float32) / 16.0
    image_shape = (8, 8)
    source = "sklearn.load_digits"

    sampled_indices = sample_balanced_indices(y, TARGET_SAMPLES, RANDOM_STATE)
    X_sampled = X[sampled_indices]
    y_sampled = y[sampled_indices]
    images_sampled = images[sampled_indices]

    np.savez_compressed(
        DIGITS_CACHE_FILE,
        X=X_sampled,
        y=y_sampled,
        images=images_sampled,
        image_shape=np.array(image_shape),
        source=np.array([source]),
    )

    return {
        "X": X_sampled,
        "y": y_sampled,
        "images": images_sampled,
        "image_shape": image_shape,
        "source": source,
    }


def extract_hog_features(images):
    features = []
    for image in images:
        feature_vector = hog(
            image,
            orientations=9,
            pixels_per_cell=(4, 4),
            cells_per_block=(1, 1),
            block_norm="L2-Hys",
            feature_vector=True,
        )
        features.append(feature_vector)
    return np.asarray(features, dtype=np.float32)


def extract_lbp_features(images):
    p = 8
    r = 1
    bins = np.arange(0, p + 3)
    features = []
    for image in images:
        lbp_image = local_binary_pattern(image, p, r, method="uniform")
        hist, _ = np.histogram(lbp_image.ravel(), bins=bins, range=(0, p + 2), density=True)
        features.append(hist)
    return np.asarray(features, dtype=np.float32)


def build_feature_matrix(images):
    hog_features = extract_hog_features(images)
    lbp_features = extract_lbp_features(images)
    combined_features = np.hstack([hog_features, lbp_features])
    return {
        "hog": hog_features,
        "lbp": lbp_features,
        "combined": combined_features,
    }


def show_sample_images(images, labels, title, image_shape, max_samples=10):
    fig, axes = plt.subplots(2, 5, figsize=(10, 4))
    for index, ax in enumerate(axes.ravel()[:max_samples]):
        ax.imshow(images[index], cmap="gray")
        ax.set_title(f"Class {labels[index]}")
        ax.axis("off")
    fig.suptitle(title)
    plt.tight_layout()
    plt.show()


def evaluate_feature_quality(feature_sets, y):
    cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    scores = {}
    for name, matrix in feature_sets.items():
        scaler = StandardScaler()
        matrix_scaled = scaler.fit_transform(matrix)
        knn = KNeighborsClassifier(n_neighbors=5)
        score = cross_val_score(knn, matrix_scaled, y, cv=cv, scoring="accuracy", n_jobs=-1).mean()
        scores[name] = score
    return scores


def knn_manual_analysis(X_train, y_train, X_test, y_test):
    cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    analysis = {}
    all_rows = []

    for metric in KNN_METRICS:
        metric_rows = []
        for k in K_VALUES:
            knn = Pipeline([
                ("scaler", StandardScaler()),
                ("knn", KNeighborsClassifier(n_neighbors=k, metric=metric)),
            ])
            cv_score = cross_val_score(knn, X_train, y_train, cv=cv, scoring="accuracy", n_jobs=-1).mean()
            start = time.perf_counter()
            knn.fit(X_train, y_train)
            train_time = time.perf_counter() - start
            start = time.perf_counter()
            y_pred = knn.predict(X_test)
            inference_time = time.perf_counter() - start
            test_accuracy = accuracy_score(y_test, y_pred)
            precision, recall, f1, _ = precision_recall_fscore_support(
                y_test, y_pred, average="weighted", zero_division=0
            )
            row = {
                "k": k,
                "metric": metric,
                "cv_accuracy": cv_score,
                "test_accuracy": test_accuracy,
                "precision": precision,
                "recall": recall,
                "f1": f1,
                "train_time": train_time,
                "inference_time": inference_time,
            }
            metric_rows.append(row)
            all_rows.append(row)
        analysis[metric] = metric_rows

    return analysis, all_rows


def format_table(rows, columns):
    headers = [header for header, _ in columns]
    widths = [len(header) for header in headers]

    normalized_rows = []
    for row in rows:
        normalized_row = []
        for index, (_, formatter) in enumerate(columns):
            value = formatter(row)
            normalized_row.append(value)
            widths[index] = max(widths[index], len(value))
        normalized_rows.append(normalized_row)

    separator = "+" + "+".join("-" * (width + 2) for width in widths) + "+"
    lines = [separator]
    lines.append("| " + " | ".join(header.ljust(widths[index]) for index, header in enumerate(headers)) + " |")
    lines.append(separator)
    for normalized_row in normalized_rows:
        lines.append("| " + " | ".join(normalized_row[index].ljust(widths[index]) for index in range(len(widths))) + " |")
    lines.append(separator)
    return "\n".join(lines)


def print_knn_performance_table(rows):
    columns = [
        ("k", lambda row: str(row["k"])),
        ("metric", lambda row: row["metric"]),
        ("cv_acc", lambda row: f'{row["cv_accuracy"]:.4f}'),
        ("test_acc", lambda row: f'{row["test_accuracy"]:.4f}'),
        ("precision", lambda row: f'{row["precision"]:.4f}'),
        ("recall", lambda row: f'{row["recall"]:.4f}'),
        ("f1", lambda row: f'{row["f1"]:.4f}'),
        ("train_s", lambda row: f'{row["train_time"]:.4f}'),
        ("infer_s", lambda row: f'{row["inference_time"]:.4f}'),
    ]
    print("\nTabel Perbandingan Performa Semua Variasi Parameter KNN")
    print(format_table(rows, columns))


def extract_svm_grid_rows(grid):
    rows = []
    results = grid.cv_results_
    for index, params in enumerate(results["params"]):
        rows.append({
            "kernel": params.get("svc__kernel", "-"),
            "C": params.get("svc__C", "-"),
            "degree": params.get("svc__degree", "-"),
            "coef0": params.get("svc__coef0", "-"),
            "gamma": params.get("svc__gamma", "-"),
```

```python
            "cv_accuracy": float(results["mean_test_score"][index]),
            "std_accuracy": float(results["std_test_score"][index]),
        })
    return rows


def print_svm_performance_table(rows):
    columns = [
        ("kernel", lambda row: str(row["kernel"])),
        ("C", lambda row: str(row["C"])),
        ("degree", lambda row: str(row["degree"])),
        ("coef0", lambda row: str(row["coef0"])),
        ("gamma", lambda row: str(row["gamma"])),
        ("cv_acc", lambda row: f'{row["cv_accuracy"]:.4f}'),
        ("cv_std", lambda row: f'{row["std_accuracy"]:.4f}'),
    ]
    print("\nTabel Perbandingan Performa Semua Variasi Parameter SVM")
    print(format_table(rows, columns))


def plot_knn_analysis(analysis):
    fig, axes = plt.subplots(1, len(KNN_METRICS), figsize=(18, 5), sharey=True)
    if len(KNN_METRICS) == 1:
        axes = [axes]

    for ax, metric in zip(axes, KNN_METRICS):
        rows = analysis[metric]
        ks = [row["k"] for row in rows]
        cv_scores = [row["cv_accuracy"] for row in rows]
        test_scores = [row["test_accuracy"] for row in rows]
        ax.plot(ks, cv_scores, marker="o", label="CV Accuracy")
        ax.plot(ks, test_scores, marker="s", label="Test Accuracy")
        ax.set_title(f"KNN Metric: {metric}")
        ax.set_xlabel("k")
        ax.set_xticks(K_VALUES)
        ax.grid(True, alpha=0.3)
    axes[0].set_ylabel("Accuracy")
    axes[0].legend()
    plt.suptitle("Pengaruh k terhadap KNN")
    plt.tight_layout()
    plt.show()


def tune_knn_gridsearch(X_train, y_train):
    knn_pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier()),
    ])
    param_grid = [
        {
            "knn__n_neighbors": K_VALUES,
            "knn__metric": ["euclidean"],
        },
        {
            "knn__n_neighbors": K_VALUES,
            "knn__metric": ["manhattan"],
        },
        {
            "knn__n_neighbors": K_VALUES,
            "knn__metric": ["minkowski"],
            "knn__p": [2, 3],
        },
    ]
    cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    grid = GridSearchCV(
        knn_pipeline,
        param_grid=param_grid,
        scoring="accuracy",
        cv=cv,
        n_jobs=-1,
        refit=True,
    )
    start = time.perf_counter()
    grid.fit(X_train, y_train)
    fit_time = time.perf_counter() - start
    return grid, fit_time


def evaluate_model(model, X_test, y_test):
    start = time.perf_counter()
    y_pred = model.predict(X_test)
    inference_time = time.perf_counter() - start
    accuracy = accuracy_score(y_test, y_pred)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test, y_pred, average="weighted", zero_division=0
    )
    report = classification_report(y_test, y_pred, digits=4, zero_division=0)
    matrix = confusion_matrix(y_test, y_pred)
    return {
        "y_pred": y_pred,
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "report": report,
        "matrix": matrix,
        "inference_time": inference_time,
    }


def plot_confusion_matrix(matrix, class_names, title):
    plt.figure(figsize=(10, 8))
    plt.imshow(matrix, cmap="Blues")
    plt.title(title)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.xticks(range(len(class_names)), class_names)
    plt.yticks(range(len(class_names)), class_names)
    plt.colorbar()

    threshold = matrix.max() / 2.0
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            color = "white" if matrix[i, j] > threshold else "black"
            plt.text(j, i, str(matrix[i, j]), ha="center", va="center", color=color)

    plt.tight_layout()
    plt.show()


def tune_svm_gridsearch(X_train, y_train):
    svm_pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("svc", SVC(probability=True, random_state=RANDOM_STATE)),
    ])
    param_grid = [
        {
            "svc__kernel": ["linear"],
            "svc__C": [0.1, 1, 10, 100],
        },
        {
            "svc__kernel": ["poly"],
            "svc__C": [0.1, 1, 10, 100],
            "svc__degree": [2, 3],
            "svc__coef0": [0.0, 1.0],
            "svc__gamma": ["scale"],
        },
        {
            "svc__kernel": ["rbf"],
            "svc__C": [0.1, 1, 10, 100],
            "svc__gamma": [0.001, 0.01, 0.1, 1],
        },
    ]
    cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    grid = GridSearchCV(
        svm_pipeline,
        param_grid=param_grid,
        scoring="accuracy",
        cv=cv,
        n_jobs=-1,
        refit=True,
    )
    start = time.perf_counter()
    grid.fit(X_train, y_train)
    fit_time = time.perf_counter() - start
    return grid, fit_time


def plot_learning_curve(estimator, X, y, title):
    cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    train_sizes, train_scores, validation_scores = learning_curve(
        estimator,
        X,
        y,
        cv=cv,
        n_jobs=-1,
        train_sizes=np.linspace(0.1, 1.0, 5),
        scoring="accuracy",
        shuffle=True,
        random_state=RANDOM_STATE,
    )

    train_mean = train_scores.mean(axis=1)
    train_std = train_scores.std(axis=1)
    val_mean = validation_scores.mean(axis=1)
    val_std = validation_scores.std(axis=1)

    plt.figure(figsize=(10, 6))
    plt.plot(train_sizes, train_mean, marker="o", label="Training Accuracy")
    plt.plot(train_sizes, val_mean, marker="s", label="Validation Accuracy")
    plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.15)
    plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.15)
    plt.title(title)
    plt.xlabel("Training Samples")
    plt.ylabel("Accuracy")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()


def plot_multiclass_roc(model, X_test, y_test, class_names, title):
    classes = np.unique(y_test)
    y_test_bin = label_binarize(y_test, classes=classes)
    y_score = model.predict_proba(X_test)

    plt.figure(figsize=(10, 8))
    for index, class_label in enumerate(classes):
        fpr, tpr, _ = roc_curve(y_test_bin[:, index], y_score[:, index])
        roc_auc = auc(fpr, tpr)
        plt.plot(fpr, tpr, lw=2, label=f"Class {class_names[class_label]} (AUC = {roc_auc:.3f})")

    plt.plot([0, 1], [0, 1], "k--", lw=2)
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(title)
    plt.legend(loc="lower right", fontsize=8)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()


def plot_pca_decision_boundary(X_train, y_train, best_svm_params, title):
    scaler = StandardScaler()
    pca = PCA(n_components=2, random_state=RANDOM_STATE)
    X_train_scaled = scaler.fit_transform(X_train)
    X_train_pca = pca.fit_transform(X_train_scaled)

    boundary_model = SVC(probability=True, random_state=RANDOM_STATE, **best_svm_params)
    boundary_model.fit(X_train_pca, y_train)

    x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
    y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, 0.05),
        np.arange(y_min, y_max, 0.05),
    )
    mesh = np.c_[xx.ravel(), yy.ravel()]
    predictions = boundary_model.predict(mesh).reshape(xx.shape)

    plt.figure(figsize=(10, 8))
    plt.contourf(xx, yy, predictions, alpha=0.25, cmap=plt.cm.tab10)
    scatter = plt.scatter(
        X_train_pca[:, 0],
        X_train_pca[:, 1],
        c=y_train,
        cmap=plt.cm.tab10,
        edgecolors="black",
        s=30,
    )
    plt.title(title)
    plt.xlabel("PCA 1")
    plt.ylabel("PCA 2")
    plt.colorbar(scatter, ticks=range(len(np.unique(y_train))))
    plt.tight_layout()
    plt.show()


def summarize_best_model(name, result):
    print(f"\n{name}")
    print("-" * len(name))
    print(f"Accuracy : {result['accuracy']:.4f}")
    print(f"Precision: {result['precision']:.4f}")
    print(f"Recall   : {result['recall']:.4f}")
    print(f"F1-score : {result['f1']:.4f}")
    print(f"Inference: {result['inference_time']:.4f} seconds")
    print("Confusion Matrix:")
    print(result["matrix"])
    print("Classification Report:")
    print(result["report"])


def main():
    print("KOMPARASI KLASIFIKASI KNN VS SVM UNTUK PENGENALAN OBJEK CITRA")
    print("=" * 72)

    dataset = load_dataset()
    X = dataset["X"]
    y = dataset["y"]
    images = dataset["images"]
    image_shape = dataset["image_shape"]
    source = dataset["source"]

    print(f"Sumber dataset : {source}")
    print(f"Jumlah sampel  : {len(X)}")
    print(f"Jumlah kelas   : {len(np.unique(y))}")
    print(f"Ukuran gambar   : {image_shape}")
    print(f"Cache dataset   : {DIGITS_CACHE_FILE}")
```



```python
    show_sample_images(images, y, "Sampel Citra dari Dataset", image_shape)

    feature_sets = build_feature_matrix(images)
    hog_shape = feature_sets["hog"].shape
    lbp_shape = feature_sets["lbp"].shape
    combined_shape = feature_sets["combined"].shape

    print(f"Fitur HOG       : {hog_shape}")
    print(f"Fitur LBP       : {lbp_shape}")
    print(f"Fitur Gabungan  : {combined_shape}")

    feature_quality = evaluate_feature_quality(feature_sets, y)
    print("\nKualitas fitur berdasarkan CV 5-fold")
    for feature_name, score in feature_quality.items():
        print(f"- {feature_name:<8}: {score:.4f}")

    X_train, X_test, y_train, y_test = train_test_split(
        feature_sets["combined"],
        y,
        test_size=TEST_SIZE,
        random_state=RANDOM_STATE,
        stratify=y,
    )

    print("\nAnalisis KNN")
    print("-" * 40)
    knn_analysis, knn_rows = knn_manual_analysis(X_train, y_train, X_test, y_test)
    print_knn_performance_table(knn_rows)
    plot_knn_analysis(knn_analysis)

    knn_grid, knn_fit_time = tune_knn_gridsearch(X_train, y_train)
    print("\nGridSearchCV KNN")
    print("-" * 40)
    print(f"Best params : {knn_grid.best_params_}")
    print(f"Best CV acc : {knn_grid.best_score_:.4f}")
    print(f"Fit time    : {knn_fit_time:.4f} seconds")

    knn_test_result = evaluate_model(knn_grid.best_estimator_, X_test, y_test)
    knn_test_result["fit_time"] = knn_fit_time
    summarize_best_model("Hasil KNN Terbaik", knn_test_result)
    plot_confusion_matrix(knn_test_result["matrix"], SVM_CLASS_NAMES, "Confusion Matrix - KNN Terbaik")
    plot_learning_curve(knn_grid.best_estimator_, X_train, y_train, "Learning Curve - KNN Terbaik")
    plot_multiclass_roc(knn_grid.best_estimator_, X_test, y_test, SVM_CLASS_NAMES, "ROC OvR - KNN Terbaik")

    print("\nAnalisis SVM")
    print("-" * 40)
    svm_grid, svm_fit_time = tune_svm_gridsearch(X_train, y_train)
    svm_rows = extract_svm_grid_rows(svm_grid)
    print_svm_performance_table(svm_rows)
    print("GridSearchCV SVM")
    print("-" * 40)
    print(f"Best params : {svm_grid.best_params_}")
    print(f"Best CV acc : {svm_grid.best_score_:.4f}")
    print(f"Fit time    : {svm_fit_time:.4f} seconds")

    svm_test_result = evaluate_model(svm_grid.best_estimator_, X_test, y_test)
    svm_test_result["fit_time"] = svm_fit_time
    summarize_best_model("Hasil SVM Terbaik", svm_test_result)
    plot_confusion_matrix(svm_test_result["matrix"], SVM_CLASS_NAMES, "Confusion Matrix - SVM Terbaik")
    plot_learning_curve(svm_grid.best_estimator_, X_train, y_train, "Learning Curve - SVM Terbaik")
    plot_multiclass_roc(svm_grid.best_estimator_, X_test, y_test, SVM_CLASS_NAMES, "ROC OvR - SVM Terbaik")

    best_svm_params = {}
    for key, value in svm_grid.best_params_.items():
        if key.startswith("svc__"):
            best_svm_params[key.replace("svc__", "")] = value

    plot_pca_decision_boundary(
        X_train,
        y_train,
        best_svm_params,
        "Decision Boundary SVM pada PCA 2D",
    )

    winner_name = "SVM" if svm_test_result["accuracy"] >= knn_test_result["accuracy"] else "KNN"
    winner_result = svm_test_result if winner_name == "SVM" else knn_test_result
    print("\nKESIMPULAN")
    print("-" * 40)
    print(f"Metode terbaik : {winner_name}")
    print(f"Akurasi test   : {winner_result['accuracy']:.4f}")
    print(f"Waktu fit KNN  : {knn_fit_time:.4f} detik")
    print(f"Waktu fit SVM  : {svm_fit_time:.4f} detik")
    print("Trade-off utama: KNN lebih sederhana dan cepat dilatih, sedangkan SVM biasanya lebih kuat saat fitur cukup representatif tetapi tuning-nya lebih mahal.")
    print("Rekomendasi: gunakan parameter terbaik dari GridSearchCV dan simpan pipeline akhir bila ingin dipakai ulang.")

    summary = {
        "dataset_source": source,
        "sample_count": int(len(X)),
        "feature_shapes": {
            "hog": hog_shape,
            "lbp": lbp_shape,
            "combined": combined_shape,
        },
        "feature_quality_cv": feature_quality,
        "knn_best_params": knn_grid.best_params_,
        "knn_best_cv_accuracy": float(knn_grid.best_score_),
        "knn_test_accuracy": float(knn_test_result["accuracy"]),
        "svm_best_params": svm_grid.best_params_,
        "svm_best_cv_accuracy": float(svm_grid.best_score_),
        "svm_test_accuracy": float(svm_test_result["accuracy"]),
        "winner": winner_name,
    }

    summary_file = BASE_DIR / "komparasi_knn_svm_summary.json"
    summary_file.write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print(f"\nRingkasan tersimpan di: {summary_file}")

    plt.show()


if __name__ == "__main__":
    main()
```





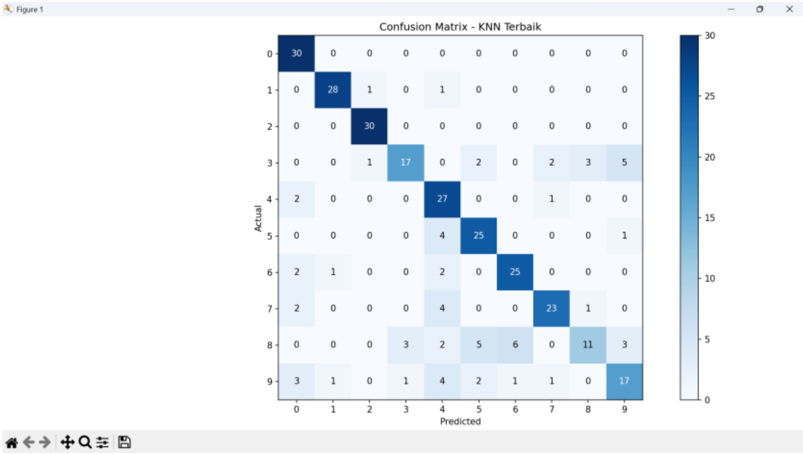


Figure 1
Confusion Matrix - KNN Terbaik
Actual
Predicted

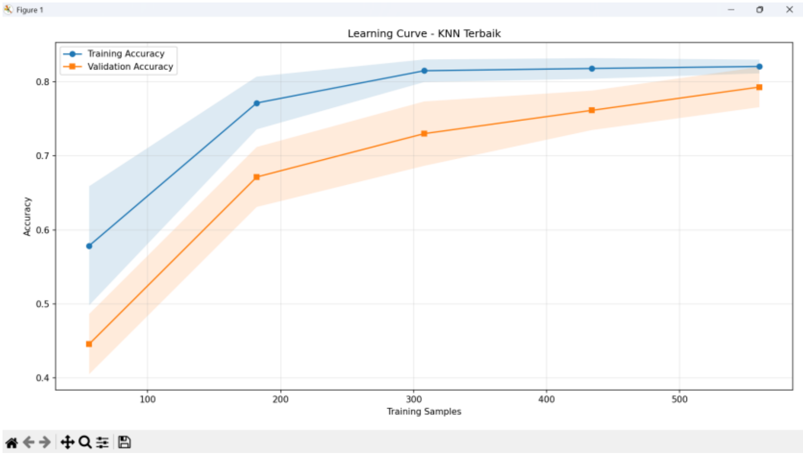


Figure 1
Learning Curve - KNN Terbaik
Training Accuracy
Validation Accuracy
Accuracy
Training Samples

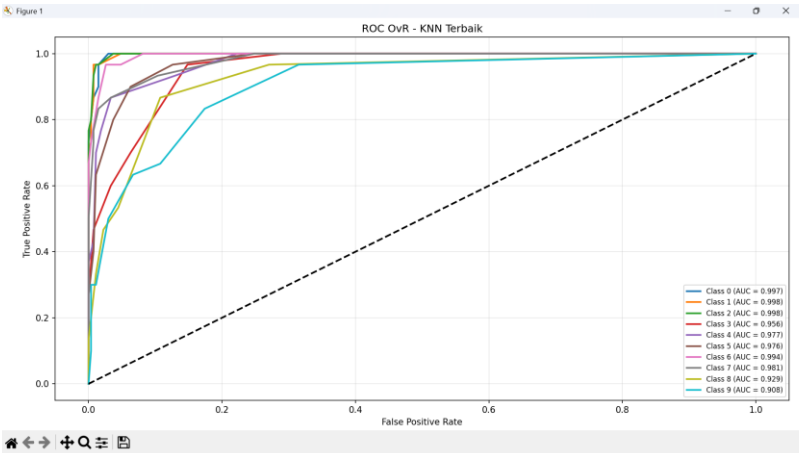


Figure 1
ROC OvR - KNN Terbaik
True Positive Rate
False Positive Rate
Class 0 (AUC = 0.997)
Class 1 (AUC = 0.998)
Class 2 (AUC = 0.998)
Class 3 (AUC = 0.956)
Class 4 (AUC = 0.977)
Class 5 (AUC = 0.976)
Class 6 (AUC = 0.994)
Class 7 (AUC = 0.981)
Class 8 (AUC = 0.929)
Class 9 (AUC = 0.908)

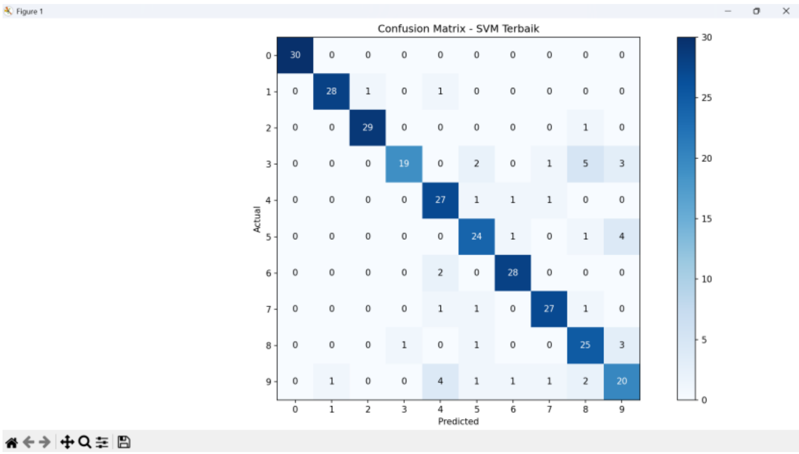


Figure 1
Confusion Matrix - SVM Terbaik
Actual
Predicted

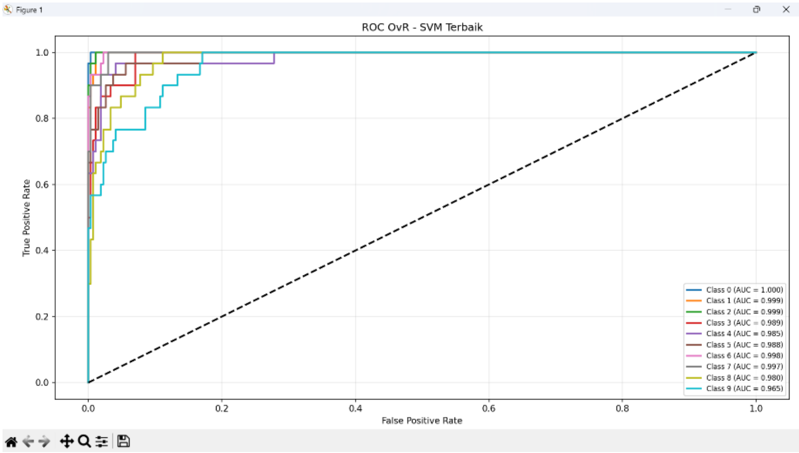


Figure 1
ROC OvR - SVM Terbaik
True Positive Rate
False Positive Rate
Class 0 (AUC = 1.000)
Class 1 (AUC = 0.999)
Class 2 (AUC = 0.999)
Class 3 (AUC = 0.989)
Class 4 (AUC = 0.985)
Class 5 (AUC = 0.988)
Class 6 (AUC = 0.998)
Class 7 (AUC = 0.997)
Class 8 (AUC = 0.980)
Class 9 (AUC = 0.965)

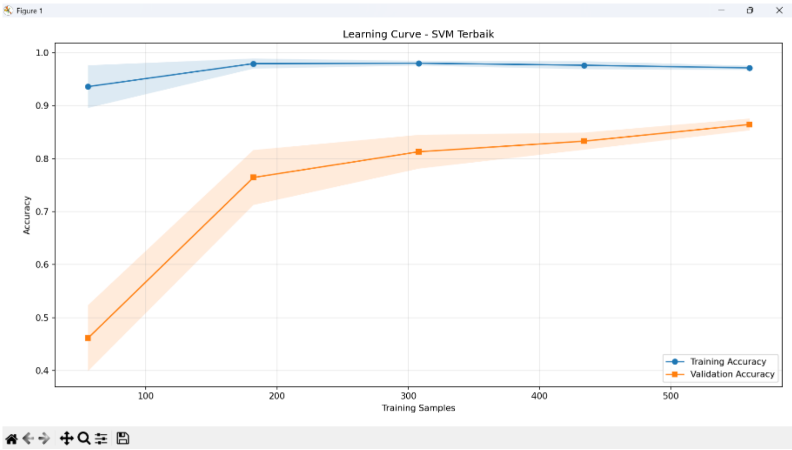


Figure 1
Learning Curve - SVM Terbaik
Accuracy
Training Samples
Training Accuracy
Validation Accuracy

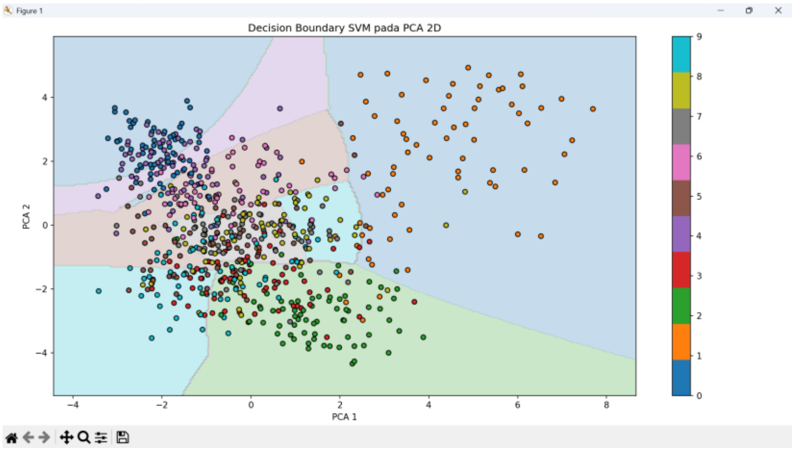


Figure 1
Decision Boundary SVM pada PCA 2D
PCA 2
PCA 1

```
KOMPARASI KLASIFIKASI KNN VS SVM UNTUK PENGENALAN OBJEK CITRA
========================================================================
Sumber dataset : sklearn.load_digits
Jumlah sampel  : 1000
Jumlah kelas   : 10
Ukuran gambar   : (8, 8)
Cache dataset   : D:\SEMESTER 4\Pengolahan Citra Digital\Pertemuan 13\dataset\digits_sample_1000.npz
Fitur HOG       : (1000, 36)
Fitur LBP       : (1000, 10)
Fitur Gabungan  : (1000, 46)

Kualitas fitur berdasarkan CV 5-fold
- hog     : 0.7530
- lbp     : 0.3150
- combined: 0.7900

Analisis KNN
========================================

Tabel Perbandingan Performa Semua Variasi Parameter KNN
+----+-----------+--------+----------+-----------+--------+--------+---------+---------+
| k  | metric    | cv_acc | test_acc | precision | recall | f1     | train_s | infer_s |
+----+-----------+--------+----------+-----------+--------+--------+---------+---------+
| 1  | euclidean | 0.7629 | 0.7633   | 0.7724    | 0.7633 | 0.7499 | 0.0060  | 1.9619  |
| 3  | euclidean | 0.7586 | 0.7767   | 0.7852    | 0.7767 | 0.7669 | 0.0080  | 0.0243  |
| 5  | euclidean | 0.7729 | 0.7800   | 0.7860    | 0.7800 | 0.7674 | 0.0058  | 0.0162  |
| 7  | euclidean | 0.7814 | 0.7900   | 0.7989    | 0.7900 | 0.7783 | 0.0035  | 0.0159  |
| 9  | euclidean | 0.7743 | 0.7800   | 0.7866    | 0.7800 | 0.7661 | 0.0077  | 0.0194  |
| 11 | euclidean | 0.7686 | 0.7567   | 0.7668    | 0.7567 | 0.7429 | 0.0067  | 0.0152  |
| 1  | manhattan | 0.7657 | 0.7500   | 0.7465    | 0.7500 | 0.7317 | 0.0039  | 0.0208  |
| 3  | manhattan | 0.7600 | 0.7667   | 0.7806    | 0.7667 | 0.7499 | 0.0067  | 0.0399  |
| 5  | manhattan | 0.7729 | 0.7733   | 0.7857    | 0.7733 | 0.7525 | 0.0070  | 0.0260  |
| 7  | manhattan | 0.7800 | 0.7833   | 0.7945    | 0.7833 | 0.7686 | 0.0070  | 0.0398  |
| 9  | manhattan | 0.7743 | 0.7867   | 0.7888    | 0.7867 | 0.7695 | 0.0069  | 0.0226  |
| 11 | manhattan | 0.7786 | 0.7833   | 0.7823    | 0.7833 | 0.7682 | 0.0067  | 0.0288  |
| 1  | minkowski | 0.7629 | 0.7633   | 0.7724    | 0.7633 | 0.7499 | 0.0069  | 0.0154  |
| 3  | minkowski | 0.7586 | 0.7767   | 0.7852    | 0.7767 | 0.7669 | 0.0075  | 0.0174  |
| 5  | minkowski | 0.7729 | 0.7800   | 0.7860    | 0.7800 | 0.7674 | 0.0070  | 0.0154  |
| 7  | minkowski | 0.7814 | 0.7900   | 0.7989    | 0.7900 | 0.7783 | 0.0056  | 0.0173  |
| 9  | minkowski | 0.7743 | 0.7800   | 0.7866    | 0.7800 | 0.7661 | 0.0056  | 0.0142  |
| 11 | minkowski | 0.7686 | 0.7567   | 0.7668    | 0.7567 | 0.7429 | 0.0068  | 0.0149  |
+----+-----------+--------+----------+-----------+--------+--------+---------+---------+
```

```
GridSearchCV KNN
----------------------------------------
Best params : {'knn__metric': 'minkowski', 'knn__n_neighbors': 11, 'knn__p': 3}
Best CV acc : 0.7929
Fit time    : 1.7421 seconds

Hasil KNN Terbaik
-----------------
Accuracy : 0.7767
Precision: 0.7819
Recall   : 0.7767
F1-score : 0.7658
Inference: 0.4351 seconds
Confusion Matrix:
[[30  0  0  0  0  0  0  0  0  0]
 [ 0 28  1  0  1  0  0  0  0  0]
 [ 0  0 30  0  0  0  0  0  0  0]
 [ 0  0  1 17  0  2  0  2  3  5]
 [ 2  0  0  0 27  0  0  1  0  0]
 [ 0  0  0  0  4 25  0  0  0  1]
 [ 2  1  0  0  2  0 25  0  0  0]
 [ 2  0  0  0  4  0  0 23  1  0]
 [ 0  0  0  3  2  5  6  0 11  3]
 [ 3  1  0  1  4  2  1  1  0 17]]
Classification Report:
              precision    recall  f1-score   support

           0     0.7692    1.0000    0.8696        30
           1     0.9333    0.9333    0.9333        30
           2     0.9375    1.0000    0.9677        30
           3     0.8095    0.5667    0.6667        30
           4     0.6136    0.9000    0.7297        30
           5     0.7353    0.8333    0.7812        30
           6     0.7812    0.8333    0.8065        30
           7     0.8519    0.7667    0.8070        30
           8     0.7333    0.3667    0.4889        30
           9     0.6538    0.5667    0.6071        30

    accuracy                         0.7767       300
   macro avg     0.7819    0.7767    0.7658       300
weighted avg     0.7819    0.7767    0.7658       300
```

```
Analysis SVM
========================================

Tabel Perbandingan Performa Semua Variasi Parameter SVM
+--------+-----+--------+-------+-------+--------+--------+
| kernel | C   | degree | coef0 | gamma | cv_acc | cv_std |
+--------+-----+--------+-------+-------+--------+--------+
| linear | 0.1 | -      | -     | -     | 0.8257 | 0.0125 |
| linear | 1   | -      | -     | -     | 0.8200 | 0.0114 |
| linear | 10  | -      | -     | -     | 0.8200 | 0.0114 |
| linear | 100 | -      | -     | -     | 0.8200 | 0.0114 |
| poly   | 0.1 | 2      | 0.0   | scale | 0.6814 | 0.0224 |
| poly   | 0.1 | 3      | 0.0   | scale | 0.6457 | 0.0439 |
| poly   | 0.1 | 2      | 1.0   | scale | 0.8471 | 0.0210 |
| poly   | 0.1 | 3      | 1.0   | scale | 0.8643 | 0.0111 |
| poly   | 1   | 2      | 0.0   | scale | 0.7829 | 0.0328 |
| poly   | 1   | 3      | 0.0   | scale | 0.8271 | 0.0360 |
| poly   | 1   | 2      | 1.0   | scale | 0.8614 | 0.0125 |
| poly   | 1   | 3      | 1.0   | scale | 0.8614 | 0.0132 |
| poly   | 10  | 2      | 0.0   | scale | 0.7757 | 0.0295 |
| poly   | 10  | 3      | 0.0   | scale | 0.8343 | 0.0311 |
| poly   | 10  | 2      | 1.0   | scale | 0.8543 | 0.0147 |
| poly   | 10  | 3      | 1.0   | scale | 0.8614 | 0.0132 |
| poly   | 100 | 2      | 0.0   | scale | 0.7757 | 0.0295 |
| poly   | 100 | 3      | 0.0   | scale | 0.8343 | 0.0311 |
| poly   | 100 | 2      | 1.0   | scale | 0.8543 | 0.0147 |
| poly   | 100 | 3      | 1.0   | scale | 0.8614 | 0.0132 |
| rbf    | 0.1 | -      | -     | 0.001 | 0.8014 | 0.0354 |
| rbf    | 0.1 | -      | -     | 0.01  | 0.8129 | 0.0368 |
| rbf    | 0.1 | -      | -     | 0.1   | 0.3814 | 0.0498 |
| rbf    | 0.1 | -      | -     | 1     | 0.1457 | 0.0340 |
| rbf    | 1   | -      | -     | 0.001 | 0.8129 | 0.0280 |
| rbf    | 1   | -      | -     | 0.01  | 0.8486 | 0.0188 |
| rbf    | 1   | -      | -     | 0.1   | 0.7714 | 0.0186 |
| rbf    | 1   | -      | -     | 1     | 0.1686 | 0.0331 |
| rbf    | 10  | -      | -     | 0.001 | 0.8443 | 0.0177 |
| rbf    | 10  | -      | -     | 0.01  | 0.8614 | 0.0086 |
| rbf    | 10  | -      | -     | 0.1   | 0.7829 | 0.0140 |
| rbf    | 10  | -      | -     | 1     | 0.1886 | 0.0453 |
| rbf    | 100 | -      | -     | 0.001 | 0.8329 | 0.0200 |
| rbf    | 100 | -      | -     | 0.01  | 0.8614 | 0.0086 |
| rbf    | 100 | -      | -     | 0.1   | 0.7829 | 0.0140 |
| rbf    | 100 | -      | -     | 1     | 0.1886 | 0.0453 |
+--------+-----+--------+-------+-------+--------+--------+
```

```
GridSearchCV SVM
----------------------------------------
Best params : {'svc__C': 0.1, 'svc__coef0': 1.0, 'svc__degree': 3, 'svc__gamma': 'scale', 'svc__kernel': 'poly'}
Best CV acc : 0.8643
Fit time    : 13.0256 seconds

Hasil SVM Terbaik
-----------------
Accuracy : 0.8567
Precision: 0.8638
Recall   : 0.8567
F1-score : 0.8561
Inference: 0.0193 seconds
Confusion Matrix:
[[30  0  0  0  0  0  0  0  0  0]
 [ 0 28  1  0  1  0  0  0  0  0]
 [ 0  0 29  0  0  0  0  0  1  0]
 [ 0  0  0 19  0  2  0  1  5  3]
 [ 0  0  0  0 27  1  1  1  0  0]
 [ 0  0  0  0  0 24  1  0  1  4]
 [ 0  0  0  0  2  0 28  0  0  0]
 [ 0  0  0  0  1  1  0 27  1  0]
 [ 0  0  0  1  0  1  0  0 25  3]
 [ 0  1  0  0  4  1  1  1  2 20]]
Classification Report:
              precision    recall  f1-score   support

           0     1.0000    1.0000    1.0000        30
           1     0.9655    0.9333    0.9492        30
           2     0.9667    0.9667    0.9667        30
           3     0.9500    0.6333    0.7600        30
           4     0.7714    0.9000    0.8308        30
           5     0.8000    0.8000    0.8000        30
           6     0.9032    0.9333    0.9180        30
           7     0.9000    0.9000    0.9000        30
           8     0.7143    0.8333    0.7692        30
           9     0.6667    0.6667    0.6667        30

    accuracy                         0.8567       300
   macro avg     0.8638    0.8567    0.8561       300
weighted avg     0.8638    0.8567    0.8561       300


KESIMPULAN
========================================
Metode terbaik : SVM
Akurasi test   : 0.8567
Waktu fit KNN  : 1.7421 detik
Waktu fit SVM  : 13.0256 detik
Trade-off utama: KNN lebih sederhana dan cepat dilatih, sedangkan SVM biasanya lebih kuat saat fitur cukup representatif tetapi tuning-nya lebih mahal.
Rekomendasi: gunakan parameter terbaik dari GridSearchCV dan simpan pipeline akhir bila ingin dipakai ulang.

Ringkasan tersimpan di: D:\SEMESTER 4\Pengolahan Citra Digital\Pertemuan 13\komparasi_knn_svm_summary.json
```